
ELYSIUM TOURS — FRONTEND DEVELOPMENT STANDARD OPERATING PROCEDURE (SOP)

Document Version: 1.0

Date: February 10, 2026

Project: Elysium Tours Web Platform

Stack: React 19 · Tailwind CSS 4 · Redux Toolkit · Axios · Vite 7

Audience: Fullstack developers (junior to mid-level) using Claude-augmented workflows

TABLE OF CONTENTS

1. Project Overview & Architecture
2. Development Environment Setup
3. Directory Structure & File Naming Conventions
4. Design System & Theming Reference
5. Component Development Standards
6. State Management with Redux
7. API Integration with Axios
8. Routing & Layout Architecture
9. Page Development Procedures
10. Claude-Augmented Development Workflow
11. Testing & Debugging Procedures
12. Git Workflow & Version Control
13. Security & Performance Standards
14. Code Review Framework (for Senior Developer)
15. Figma Design Reference Guide
16. Appendices

1. PROJECT OVERVIEW & ARCHITECTURE

1.1 What is Elysium Tours?

Elysium Tours is a tourism web platform focused on Ghana. The platform allows tourists to discover tours, browse destinations, read blog content, view galleries, and connect with tour partners (accommodation providers, transport companies, tour guides, restaurants, photographers, and insurance providers). There is also an admin interface for data management.

1.2 Two User Roles

The platform has two distinct interface contexts:

Tourist-Facing (Public): The main website that tourists interact with — browsing tours, reading blogs, viewing galleries, contacting support, and booking.

Admin-Facing (Internal): A dashboard for administrators to upload tour data, manage partner listings, handle blog content, view messages, manage newsletter subscriptions, and handle support tickets.

1.3 Technology Stack Summary

Technology	Version	Purpose
React	19.1	UI library (functional components with hooks)
Vite	7.x	Build tool and dev server
Tailwind CSS	4.x	Utility-first styling (via @tailwindcss/vite plugin)
Redux Toolkit	via react-redux 9.2	Global state management
Redux Persist	6.x	Persists Redux state across sessions
Redux Thunk	3.x	Async action middleware
Axios	1.11	HTTP client for API calls
React Router	7.x + react-router-dom 6.x	Client-side routing
Universal Cookie	8.x	Cookie management (auth tokens)

1.4 Architecture Diagram (Mental Model)

```
Browser
├── React App (Vite Dev Server / Build)
│   ├── Router (react-router-dom)
│   │   ├── TouristLayout → Public Pages
│   │   └── AdminLayout → Admin Pages (Protected)
│   ├── Redux Store
│   │   ├── Slices (auth, tours, blogs, etc.)
│   │   └── Thunks → Axios Service Calls → Backend API
│   └── Tailwind CSS (theme tokens from App.css @theme)
```

1.5 Design Viewport

All Figma designs are at **1728px width** (desktop). Responsive breakpoints will need to be implemented for tablet and mobile. The Figma file currently has empty Tablet and Mobile pages, so desktop-first development is the priority.

2. DEVELOPMENT ENVIRONMENT SETUP

2.1 Prerequisites

Before you begin, ensure you have these installed on your machine:

- **Node.js** v18 or higher (check with `node -v`)
- **npm** v9 or higher (comes with Node.js; check with `npm -v`)
- **Git** (check with `git --version`)
- **VS Code** (recommended editor)
- **VS Code Extensions:** ESLint, Tailwind CSS IntelliSense, Prettier, ES7+ React/Redux snippets

2.2 Clone and Install

```
# Step 1: Clone the repository
git clone <repository-url> elysiumtours-frontend
cd elysiumtours-frontend

# Step 2: Install dependencies
npm install

# Step 3: Create your .env file
# Copy the example and fill in your values
cp .env.example .env
```

2.3 Environment Variables

Create a `.env` file in the project root with at minimum:

```
VITE_API_BASE_URL=http://localhost:5000/api
VITE_APP_NAME=Elysium Tours
```

Important: All environment variables in Vite must be prefixed with `VITE_` to be accessible in the browser. Access them with `import.meta.env.VITE_API_BASE_URL`.

2.4 Running the Development Server

```
# Start the dev server (hot-reload enabled)
npm run dev

# Build for production
npm run build

# Preview production build locally
npm run preview

# Run ESLint
npm run lint
```

2.5 Understanding the Build Pipeline

The project uses **Tailwind CSS v4** with the `@tailwindcss/vite` plugin. This means:

- Tailwind is processed as a Vite plugin (NOT PostCSS in this setup)
- Theme tokens are defined in `src/App.css` inside a `@theme { }` block

- You import Tailwind with `@import "tailwindcss";` at the top of App.css
- The `tailwind.config.js` file extends the default theme with CSS custom properties

How styling flows:

1. `App.css` defines CSS custom properties inside `@theme { }`
2. `tailwind.config.js` maps those CSS variables to Tailwind utility class names
3. Components use Tailwind classes like `bg-secondary-normal-default` which resolve to the CSS variables

3. DIRECTORY STRUCTURE & FILE NAMING CONVENTIONS

3.1 Target Directory Structure

This is the complete directory structure you should build toward. Items marked with ■ already exist; items marked with ■ need to be created.

```

elysiumtours-frontend/
■■■■ .env ■
■■■■ .env.example ■
■■■■ .gitignore ■
■■■■ index.html ■
■■■■ package.json ■
■■■■ vite.config.js ■
■■■■ tailwind.config.js ■
■■■■ postcss.config.js ■
■■■■ eslint.config.js ■
■■■■ public/ ■
■ ■■■■ favicon.ico ■
■
■■■■ src/
■■■■ main.jsx ■
■■■■ App.jsx ■ (needs refactoring – currently a demo page)
■■■■ App.css ■ (theme tokens defined here)
■
■■■■ assets/ ■
■ ■■■■ images/ ■ (organize images by feature)
■ ■■■■ icons/ ■ (SVG icon components)
■ ■■■■ breadcrumbIcon.svg ■
■
■■■■ components/
■ ■■■■ ui/ ■ (reusable micro-components)
■ ■ ■■■■ Button.jsx ■
■ ■ ■■■■ Filter.jsx ■
■ ■ ■■■■ Search.jsx ■
■ ■ ■■■■ Pagination.jsx ■
■ ■ ■■■■ Breadcrumb.jsx ■
■ ■ ■■■■ Toast.jsx ■ (empty – needs implementation)
■ ■ ■■■■ TextInput.jsx ■
■ ■ ■■■■ SelectInput.jsx ■
■ ■ ■■■■ Textarea.jsx ■
■ ■ ■■■■ Label.jsx ■
■ ■ ■■■■ Checkbox.jsx ■
■ ■ ■■■■ RadioGroup.jsx ■
■ ■ ■■■■ Modal.jsx ■
■ ■ ■■■■ Switch.jsx ■
■ ■ ■■■■ Tabs.jsx ■
■ ■ ■■■■ Card.jsx ■
■ ■ ■■■■ Loader.jsx ■

```

```

■ ■ ■■ Dropdown.jsx ■
■ ■ ■■ DatePicker.jsx ■
■ ■ ■■ TimePicker.jsx ■
■ ■ ■■ FileUpload.jsx ■
■ ■
■ ■■ cards/ ■ (specialized card components)
■ ■ ■■ PopularTourCard.jsx
■ ■ ■■ TestimonialCard.jsx
■ ■ ■■ FeaturedDestinationCard.jsx
■ ■ ■■ PartnerHighlightCard.jsx
■ ■ ■■ TourTypeCard.jsx
■ ■ ■■ CountryTourCard.jsx
■ ■
... (truncated)

```

3.2 Naming Conventions

Item	Convention	Example
Component files	PascalCase .jsx	PopularTourCard.jsx
Non-component JS	camelCase .js	authService.js, useAuth.js
Redux slices	camelCase with slice suffix	tourSlice.js
Custom hooks	camelCase with use prefix	useAuth.js, useDebounce.js
Service files	camelCase with service suffix	tourService.js
CSS files	kebab-case or PascalCase matching component	App.css
Folders	camelCase	tourPartners/, auth/
Constants	UPPER_SNAKE_CASE	API_BASE_URL, MAX_FILE_SIZE
Component names	PascalCase (matches file name)	export default PopularTourCard
Props	camelCase	isActive, onClick, className
Event handlers	handle + Event	handleSubmit, handleFilterChange
Boolean props	is/has/should prefix	isLoading, hasError, isActive

3.3 Import Order Convention

Always order your imports in this sequence inside every file:

```

// 1. React and framework imports
import React, { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import { useDispatch, useSelector } from 'react-redux';

// 2. Redux slices and actions
import { fetchTours } from '../../redux/tours/tourSlice';

// 3. Services
import tourService from '../../services/tourService';

// 4. Custom hooks
import { useDebounce } from '../../hooks/useDebounce';

// 5. Components (shared first, then local)

```

```
import Button from '../ui/Button';
import Card from '../ui/Card';
import PopularTourCard from '../cards/PopularTourCard';

// 6. Utilities and constants
import { classNames } from '../../utils/classNames';
import { TOUR_CATEGORIES } from '../../utils/constants';

// 7. Assets (icons, images)
import { SearchIcon } from '../../utils/assets';
```

4. DESIGN SYSTEM & THEMING REFERENCE

4.1 Color Palette

The design uses three color families, each with states (Light, Normal, Dark) and sub-states (default, hover, active). These are defined in `App.css` inside the `@theme` block.

Primary (White/Neutral — 60% usage): Backgrounds, surfaces, neutral text.

- `primary-light-default` → `#fefefe` (page backgrounds)
- `primary-normal-default` → `#f7f7f7` (section backgrounds)
- `primary-dark-default` → `#b9b9b9` (disabled text, borders)
- `primary-dark-darker` → `#565656` (secondary text)

Secondary (Violet — 30% usage): Brand color. CTAs, active states, links.

- `secondary-light-default` → `#f2eaf9` (light violet backgrounds)
- `secondary-normal-default` → `#7b2cbf` (buttons, active filters, links)
- `secondary-normal-hover` → `#6f28ac` (hover states)
- `secondary-normal-active` → `#622399` (pressed states)
- `secondary-dark-default` → `#5c218f` (active breadcrumbs)
- `secondary-dark-darker` → `#2b0f43` (very dark violet)

Tertiary (Dark/Black — 10% usage): Body text, headings, footer.

- `tertiary-normal-default` → `#2d2d2d` (body text)
- `tertiary-dark-default` → `#222222` (footer background)
- `tertiary-dark-darker` → `#101010` (deepest black)

Usage in Tailwind:

```
<div className="bg-secondary-normal-default">                                {/* #7b2cbf */}
<p className="text-tertiary-normal-default">                                {/* #2d2d2d */}
<button className="bg-secondary-normal-default hover:bg-secondary-normal-hover active:bg-secondary-normal-active">
```

4.2 Typography Scale

Font: **Raleway** (Google Fonts). Scale factor: 1.25 (Major Third). Base: 16px.

Token	Size	Use
<code>text-sm</code>	10px	Captions, fine print

Token	Size	Use
text-med-small	13px	Small labels, metadata
text-md	16px	Body text (base)
text-semi-md	20px	Subtitles
text-High-md	25px	Section subtitles
text-Display-small	31px	Section headings
text-Display-md-small	39px	Page sub-headings
text-Display-md	49px	Page headings
text-Display-xl	56px	Hero headings (Home banner uses 56px bold)
text-Display-2xl	61px	Large hero text
text-Display-3xl	76px	Display text

Weight variants per size: Regular(400), Medium(500), Semibold(600), Bold(700), ExtraBold(800).

4.3 Spacing, Radius, and Shadows

Token	Value	Usage
radius-sm	10px	Cards, inputs
radius-md	40px	Buttons, pills
radius-lg	100px	Avatars, full-round
padding-x-sm / padding-y-sm	10px	Default button/filter padding
.shadow	0px 4px 4px rgba(0,0,0,0.05)	Subtle elevation
.shadow-lg	0px 4px 20px rgba(0,0,0,0.05)	Card elevation

4.4 Grid Layout

Designs are at **1728px** width. Use a container with side padding:

```
<div className="max-w-[1728px] mx-auto px-[30px]">
  <div className="grid grid-cols-12 gap-4">{/* content */}</div>
</div>
```

5. COMPONENT DEVELOPMENT STANDARDS

5.1 The Component Pattern

All existing components follow this exact structure. **Replicate this for every new component:**

```
import React from "react";
import { classNames } from '../utils/classNames';

const ComponentName = React.forwardRef(({
  children,
  variant = "default",
  size = "medium",
  disabled = false,
  className = "",
  onClick,
  ...props
}, ref) => {
  const baseStyles = 'rounded-md shadow transition-all duration-300 ease-in';
  const variants = { default: '...', active: '...' };
  const sizes = { small: '...', medium: '...' };
  const disabledStyles = 'opacity-50 cursor-not-allowed';

  return (
    <element
      ref={ref}
      className={classNames(baseStyles, variants[variant], sizes[size], disabled ? disabledStyles : '', className)}
      onClick={onClick}
      disabled={disabled}
      {...props}
    >
      {children}
    </element>
  );
});
ComponentName.displayName = "ComponentName";
export default ComponentName;
```

5.2 Key Patterns (Mandatory)

1. **React.forwardRef** — All UI components must use it.
2. **classNames() utility** — Never string-concatenate classes.
3. **Variant objects** — Define style variants as plain objects, select via `variants[variant]`.
4. **...props spread** — Always spread remaining props on root element.
5. **className last** — Consumer-passed className goes last in `classNames()` for overrides.
6. **Controlled + Uncontrolled** — Stateful components (inputs, filters) support both:


```
const isControlled = controlledValue !== undefined;
const currentValue = isControlled ? controlledValue : internalValue;
```
7. **Compound components** — Related components export from one file (e.g., `Filter` + `FilterOption`).
8. **Transition classes** — Use `transition-all duration-300 ease-in` for all interactive elements.

5.3 Micro UI Component Checklist

Component	Variants	States
Button	secondary, secondaryOutline, neutral	default, hover, active, disabled
TextInput	default, error, success	default, focus, filled, error, disabled
SelectInput	default, error	default, open, selected, error, disabled
TextArea	default, error	default, focus, filled, error, disabled

Component	Variants	States
Checkbox	—	unchecked, checked, disabled
RadioGroup	—	unselected, selected, disabled
Modal	—	open, closed
Switch/Toggle	—	off, on, disabled
Toast	success, error, warning, info	visible, hidden
Tabs	—	active, inactive
Loader	skeleton, spinner	loading
Card	default, elevated	default, hover
Dropdown	default	closed, open, selected
FileUpload	drag, browse	empty, uploading, uploaded, failed
DatePicker	—	closed, open, selected
TimePicker	—	closed, open, selected

5.4 New Component Procedure

1. Identify the component in Figma (Section 15 has node IDs)
2. List all visual states
3. Create the file in the correct directory (Section 3.1)
4. Write the component following the pattern in 5.1
5. Define all variant/size style objects
6. Test every variant and state visually
7. Verify transitions work (transition-all duration-300 ease-in)

6. STATE MANAGEMENT WITH REDUX

6.1 Store Setup

```
// src/redux/store.js
import { configureStore } from '@reduxjs/toolkit';
import { persistStore, persistReducer } from 'redux-persist';
import storage from 'redux-persist/lib/storage';
import { combineReducers } from 'redux';
import authReducer from '../auth/authSlice';
import tourReducer from '../tours/tourSlice';
// ... import all other reducers

const persistConfig = { key: 'elysium-root', storage, whitelist: ['auth'] };

const rootReducer = combineReducers({
  auth: authReducer,
  tours: tourReducer,
  // ... all other reducers
});
```

```

});

const persistedReducer = persistReducer(persistConfig, rootReducer);

export const store = configureStore({
  reducer: persistedReducer,
  middleware: (getDefaultMiddleware) => {
    getDefaultMiddleware({
      serializableCheck: { ignoredActions: ['persist/PERSIST', 'persist/REHYDRATE'] },
    }),
  });
export const persistor = persistStore(store);

```

6.2 Slice Pattern (Template for Every Slice)

```

// src/redux/tours/tourSlice.js
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import tourService from '../../services/tourService';

export const fetchTours = createAsyncThunk(
  'tours/fetchTours',
  async (params, { rejectWithValue }) => {
    try {
      const response = await tourService.getAll(params);
      return response.data;
    } catch (error) {
      return rejectWithValue(error.response?.data?.message || 'Failed to fetch tours');
    }
  }
);

const tourSlice = createSlice({
  name: 'tours',
  initialState: {
    items: [],
    selectedItem: null,
    isLoading: false,
    error: null,
    pagination: { page: 1, totalPages: 1, totalItems: 0 },
  },
  reducers: {
    clearError: (state) => { state.error = null; },
    clearSelectedItem: (state) => { state.selectedItem = null; },
    setPage: (state, action) => { state.pagination.page = action.payload; },
  },
  extraReducers: (builder) => {
    builder
      .addCase(fetchTours.pending, (state) => { state.isLoading = true; state.error = null; })
      .addCase(fetchTours.fulfilled, (state, action) => {
        state.isLoading = false;
        state.items = action.payload.data;
        state.pagination = action.payload.pagination;
      })
      .addCase(fetchTours.rejected, (state, action) => {
        state.isLoading = false;
        state.error = action.payload;
      });
  },
});

export const { clearError, clearSelectedItem, setPage } = tourSlice.actions;
export default tourSlice.reducer;

```

6.3 Custom Hook Pattern (Template for Every Slice)

```
// src/redux/tours/useTour.js
import { useSelector, useDispatch } from 'react-redux';
import { fetchTours, clearError, clearSelectedItem, setPage } from './tourSlice';

export const useTour = () => {
  const dispatch = useDispatch();
  const { items, selectedItem, isLoading, error, pagination } = useSelector((state) => state.tours);
  return {
    tours: items, selectedTour: selectedItem, isLoading, error, pagination,
    getTours: (params) => dispatch(fetchTours(params)),
    clearError: () => dispatch(clearError()),
    clearSelectedItem: () => dispatch(clearSelectedItem()),
    setPage: (page) => dispatch(setPage(page)),
  };
};
```

6.4 All Required Slices

Slice Directory	Slice File	Hook File	Purpose
redux/auth/	authSlice.js	useAuth.js	Login, register, tokens, user profile
redux/tours/	tourSlice.js	useTour.js	Tour listings, detail, search
redux/testimonials/	testimonialSlice.js	useTestimonial.js	Customer testimonials
redux/blogs/	blogSlice.js	useBlog.js	Blog posts by category
redux/destinations/	destinationSlice.js	useDestination.js	Featured destinations
redux/faqs/	faqSlice.js	useFaq.js	FAQ items
redux/reviews/	reviewSlice.js	useReview.js	Tour reviews
redux/payment/	paymentSlice.js	usePayment.js	Payment processing
redux/partners/	partnerSlice.js	usePartner.js	Tour partners (all types)
redux/gallery/	gallerySlice.js	useGallery.js	Photos and videos
redux/newsletter/	newsletterSlice.js	useNewsletter.js	Newsletter subscription

7. API INTEGRATION WITH AXIOS

7.1 Axios Instance

```
// src/services/api.js
import axios from 'axios';
import Cookies from 'universal-cookie';
const cookies = new Cookies();

const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
```

```

    timeout: 15000,
    headers: { 'Content-Type': 'application/json' },
  });

  // Attach auth token to every request
  api.interceptors.request.use((config) => {
    const token = cookies.get('accessToken');
    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
  }, (error) => Promise.reject(error));

  // Handle 401 globally
  api.interceptors.response.use(
    (response) => response,
    (error) => {
      if (error.response?.status === 401) {
        cookies.remove('accessToken', { path: '/' });
        cookies.remove('refreshToken', { path: '/' });
        window.location.href = '/login';
      }
      return Promise.reject(error);
    }
  );
}
export default api;

```

7.2 Service File Pattern

```

// src/services/tourService.js
import api from './api';
const tourService = {
  getAll: (params) => api.get('/tours', { params }),
  getById: (id) => api.get(`/tours/${id}`),
  create: (data) => api.post('/tours', data),
  update: (id, data) => api.put(`/tours/${id}`, data),
  delete: (id) => api.delete(`/tours/${id}`),
};
export default tourService;

```

7.3 All Required Services

Service File	Endpoint	Key Methods
authService.js	/auth	login, register, verifyOtp, forgotPassword, resetPassword, activateAccount, logout
tourService.js	/tours	getAll, getById, getPopular, getByCountry, getByType
testimonialService.js	/testimonials	getAll
blogService.js	/blogs	getAll, getById, getByCategory
contactService.js	/contact	submitForm
newsletterService.js	/newsletter	subscribe
reviewService.js	/reviews	getByTourId, create
paymentService.js	/payments	createPayment, verifyPayment
partnerService.js	/partners	getAll, getById, getByCategory

Service File	Endpoint	Key Methods
transportService.js	/transport	getAll, getByld
accommodationService.js	/accommodation	getAll, getByld
diningService.js	/dining	getAll, getByld
galleryService.js	/gallery	getAll, getCategory, uploadImage, uploadVideo

8. ROUTING & LAYOUT ARCHITECTURE

8.1 Route Structure

```
// src/routes/AppRoutes.jsx
import React from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import TouristLayout from '../layout/TouristLayout';
import AdminLayout from '../layout/AdminLayout';
import ProtectedRoute from '../ProtectedRoute';

// Tourist Pages
import HomePage from '../pages/tourist/HomePage';
import ToursPage from '../pages/tourist/ToursPage';
import TourDetailPage from '../pages/tourist/TourDetailPage';
import BlogPage from '../pages/tourist/BlogPage';
import SingleBlogPage from '../pages/tourist/SingleBlogPage';
import GalleryPage from '../pages/tourist/GalleryPage';
import TourPartnersPage from '../pages/tourist/TourPartnersPage';
import TourPartnerProfilePage from '../pages/tourist/TourPartnerProfilePage';
import ContactUsPage from '../pages/tourist/ContactUsPage';
import NotFoundPage from '../pages/tourist/NotFoundPage';

// Auth Pages
import LoginPage from '../pages/auth/LoginPage';
import SignUpPage from '../pages/auth/SignUpPage';

// Admin Pages
import DashboardOverviewPage from '../pages/admin/DashboardOverviewPage';

const AppRoutes = () => {
  <BrowserRouter>
  <Routes>
    {/* PUBLIC TOURIST ROUTES */}
    <Route element={<TouristLayout />}>
      <Route path="/" element={<HomePage />} />
      <Route path="/tours" element={<ToursPage />} />
      <Route path="/tours/:id" element={<TourDetailPage />} />
      <Route path="/blog" element={<BlogPage />} />
      <Route path="/blog/:id" element={<SingleBlogPage />} />
      <Route path="/gallery" element={<GalleryPage />} />
      <Route path="/tour-partners" element={<TourPartnersPage />} />
      <Route path="/tour-partners/:id" element={<TourPartnerProfilePage />} />
      <Route path="/contact" element={<ContactUsPage />} />
    </Route>
    {/* AUTH ROUTES (no layout wrapper) */}
```

```
<Ro
... (truncated)
```

8.2 Protected Route

```
// src/routes/ProtectedRoute.jsx
import React from 'react';
import { Navigate, Outlet, useLocation } from 'react-router-dom';
import { useAuth } from '../redux/auth/useAuth';

const ProtectedRoute = () => {
  const { isAuthenticated, user } = useAuth();
  const location = useLocation();
  if (!isAuthenticated) return <Navigate to="/login" state={{ from: location }} replace />;
  if (user?.role !== 'admin') return <Navigate to="/" replace />;
  return <Outlet />;
};
export default ProtectedRoute;
```

8.3 Layout Components

```
// src/layout/TouristLayout.jsx
import React from 'react';
import { Outlet } from 'react-router-dom';
import Navbar from '../components/shared/Navbar';
import Footer from '../components/shared/Footer';

const TouristLayout = () => (
  <div className="min-h-screen flex flex-col">
    <Navbar />
    <main className="flex-1"><Outlet /></main>
    <Footer />
  </div>
);
export default TouristLayout;

// src/layout/AdminLayout.jsx – sidebar + outlet
import React from 'react';
import { Outlet } from 'react-router-dom';

const AdminLayout = () => (
  <div className="flex min-h-screen">
    <aside className="w-64 bg-tertiary-dark-default text-primary-light-default p-4">
      {/* Sidebar navigation links */}
    </aside>
    <main className="flex-1 p-6 bg-primary-normal-default"><Outlet /></main>
  </div>
);
export default AdminLayout;
```

9. PAGE DEVELOPMENT PROCEDURES

9.1 Page Structure Pattern

A page is a composition of sections. Pages contain minimal logic — they import and render section components:

```
// src/pages/tourist/HomePage.jsx
import React from 'react';
import HomeBanner from '../../components/sections/home/HomeBanner';
import PopularTourSection from '../../components/sections/home/PopularTourSection';
import FeaturedDestinationSection from '../../components/sections/home/FeaturedDestinationSection';
import TestimonialSection from '../../components/sections/home/TestimonialSection';
import PartnerHighlightsSection from '../../components/sections/home/PartnerHighlightsSection';
import PartnerWithUsSection from '../../components/sections/home/PartnerWithUsSection';
import SignUpBookNowSection from '../../components/sections/home/SignUpBookNowSection';

const HomePage = () => (
  <>
    <HomeBanner />
    <PopularTourSection />
    <FeaturedDestinationSection />
    <TestimonialSection />
    <PartnerHighlightsSection />
    <PartnerWithUsSection />
    <SignUpBookNowSection />
  </>
);
export default HomePage;
```

9.2 All Pages and Their Sections (from Figma Analysis)

HomePage (1728x7760px) — Figma node 134:1481

#	Section Component	Figma Frame	Description
1	Navbar	Frame 1	Logo + nav links (Home, Tours, Blog, Tour Partners, Gallery) + "Contact Us" button
2	HomeBanner	Frame 6	Hero image carousel, "Discover Ghana with Elysium Tours" (56px bold), search/filter
3	IntroSection	Frame 7	"DISCOVER GHANA LIKE NEVER BEFORE" tagline (13px bold), description (25px bold), stat counters
4	PopularTourSection	Frame 8	Section headline + 4x PopularTourCards + "View All" button
5	FeaturedDestinationSection	Frame 9	Section headline + destination image grid (2-column layout with overlay text)
6	TestimonialSection	Frame 11	Section headline + testimonial cards + "View All" button

#	Section Component	Figma Frame	Description
7	PartnerHighlightsSection	Frame 10	Section headline + 3x PartnerHighlightCards + "View All" button
8	PartnerWithUsSection	Frame 12	CTA for partners to join
9	SignUpBookNowSection	Frame 13	Section headline + 3 feature blocks
10	Footer	Frame 74/75	Logo, link columns, newsletter signup, social media, copyright

ToursPage (1728x6410px) — Figma node 186:1274

#	Section Component	Figma Frame	Description
1	TourSectionHero	Frame 102	Hero image with overlay text
2	TourByCountrySection	Frame 7/8	Country tour cards in grid
3	TypesOfToursSection	Frame 103	Tour type cards: Day Tours, Multi-Day, Festivals, Heritage, Nature, Eco & Cultural
4	NewsletterSection	Frame 108	Subscribe CTA

BlogPage (1728x8565px) — Figma node 244:1018

#	Section Component	Figma Frame	Description
1	BlogHero	Frame 111	Hero section
2	TravelGuidesSection	Frame 8	Featured travel guides with cards
3	ContentListSection	Frame 109	7 sub-sections: Travel Guides, Destination Highlights, Behind the Scenes, Festival Calendar, Diaspora Stories, Partner Spotlight, Photo & Video Stories
4	SubscribeToNewsletter	Frame 110	Newsletter CTA

ContactUsPage (1728x5359px) — Figma node 370:1829

#	Section Component	Figma Frame	Description
1	FaqsSection	Frame 109	Accordion FAQ cards (expandable)

#	Section Component	Figma Frame	Description
2	ContactFormSection	Frame 184	Name, email, phone, subject, message form + Google Maps embed
3	HelpWithBooking	Frame 108	Booking assistance CTA

TourPartnersPage (1728x10881px) — Figma node 969:6651

#	Section Component	Figma Frame	Description
1	TourPartnersHero	Frame 108	Hero banner
2	PartnerCategoryListing	Frame 3071	8 partner categories: Tour Sites & Events, Accommodation, Transportation, Tour Guides, Restaurants & Dining, Photo & Videographers, Insurance

Tour Partner Profile Sub-Pages (all ~1728x3937px)

These are individual category pages within the Tour Partners section. Each follows an identical layout pattern:

Figma Node	Page Name	Card Type
946:6216	Tour Sites & Events	SiteEventCard
946:6956	Accommodation	AccommodationCard
946:7301	Transportation	TransportationCard
946:8003	Tour Guides	TourGuideCard
946:8313	Restaurants & Dining	DiningCard
946:8693	Photo & Videographers	PhotoVideoCard
946:9047	Insurance & Other Services	InsuranceCard

Each sub-page has: Navbar → Breadcrumb → Section Headline → Filter/Sort Bar → Card Grid → Pagination → Footer. **These are highly reusable** — build one template and parameterize it.

GalleryPage (1728x11656px) — Figma node 564:2376

#	Section Component	Description
1	Gallery filter tabs	8 categories: All, Destination, Activities, Nature, Culture, Video, Partner, Travel Highlights
2	GalleryGrid	Responsive image/video grid
3	ImageViewer modal	Full-screen overlay with prev/next, share, download
4	VideoPlayer	Video playback with speed controls (0.5x, 1x, 1.5x, 2x)

#	Section Component	Description
5	ShareModal	Social sharing (link copy, social media)
6	UploadModal	User photo/video upload form with drag-and-drop

Auth Pages (no Figma frames visible — use standard auth layout)

Page	Components
LoginPage	AuthAside (brand imagery) + AuthMain (email + password form)
SignUpPage	AuthAside + AuthMain (name, email, password, confirm password)
ForgotPasswordPage	AuthAside + AuthMain (email input)
ResetPasswordPage	AuthAside + AuthMain (new password + confirm)
OtpVerificationPage	AuthAside + AuthMain (OTP code input)
AccountActivatePage	AuthAside + AuthMain (success message + CTA)

NotFoundPage (1728x1169px) — Figma node [395:1828](#)

Simple 404 page with illustration, "Page doesn't exist" message, and "Go Home" button.

Admin Pages (no Figma frames — use standard dashboard layout)

Page	Description
DashboardOverviewPage	Stats cards, recent activity
TourDataUploadPage	CRUD table + form for tours
PartnerDataUploadPage	CRUD table + form for partners
BlogDataPage	CRUD table + form for blog posts
MessagePage	List of contact form submissions
NewsletterSubscriptionDataPage	List of subscribers
SupportAndTicketPage	Support tickets list
TourSubscriptionDataPage	Tour booking subscriptions

10. CLAUDE-AUGMENTED DEVELOPMENT WORKFLOW

10.1 The Development Cycle (Follow This Exactly)

Step 1: RECEIVE TASK

- Check the Task Tracker spreadsheet for your assigned task
- Read the task description, priority, and acceptance criteria

Step 2: PROMPT CLAUDE FOR IMPLEMENTATION PLAN

- Use the template in Section 10.2 below
- Include: task name, file paths, Figma node ID, design tokens

Step 3: AUDIT THE PLAN

- Does the plan create files in the correct directories? (check Section 3.1)
- Does the plan follow component patterns? (check Section 5.1)
- Does the plan follow naming conventions? (check Section 3.2)
- Is the plan complete? (all states, variants, error handling)

Step 4: REFINE UNTIL CORRECT

- If any audit item fails, tell Claude what needs to change
- Reference specific SOP sections in your refinement prompts

Step 5: IMPLEMENT (Multi-Phase)

- Phase 1: Create file structure and boilerplate
- Phase 2: Implement UI markup and Tailwind styling
- Phase 3: Wire up Redux state / API calls (if needed)
- Phase 4: Add interactive states, transitions, event handlers
- Phase 5: Edge cases – loading, error, empty states

Step 6: TEST

- Manual: Run `npm run dev`, visually inspect in browser
- Check all variants, hover states, click states, disabled states
- Check console for errors/warnings
- Let Claude write unit tests if time permits

Step 7: DEBUG (if issues found)

- Prompt Claude: "Add console.log in [file] to trace [issue]"
- Check Redux DevTools for state issues
- Check Network tab for API issues

Step 8: FIX AND CONTINUE

- Fix the bug, remove debug logs, re-test

Step 9: COMMIT AND PUSH

- git add [specific files]
- git commit -m "feat(scope): description"
- git push origin feat/your-branch
- RULE: Small, frequent pushes. One logical change per commit.

Step 10: UPDATE TASK TRACKER

- Mark task status as "In Review"
- Senior developer reviews using Section 14 framework

10.2 Prompt Template for Implementation Plans

Copy this template and fill in the bracketed sections:

I'm working on the Elysium Tours frontend (React 19, Tailwind CSS 4, Redux, Axios, Vite 7).

TASK: [e.g., "Build the PopularTourSection component"]

CONTEXT:

- Component pattern: React.forwardRef, classNames() utility, variant objects, ...props spread
- Tailwind uses custom CSS variable tokens (e.g., bg-secondary-normal-default → #7b2cbf)
- Figma node ID: [e.g., 134:1481, Frame 8]
- This section shows: [describe what you see in the Figma design]

FILES TO CREATE/MODIFY:

- [e.g., src/components/sections/home/PopularTourSection.jsx]
- [e.g., src/components/cards/PopularTourCard.jsx]

REQUIREMENTS:

- [List specific requirements]

-
- Follow project component patterns (forwardRef, classNames, variant objects)
 - Handle loading, error, and empty states
 - Use transition-all duration-300 ease-in for interactive elements

Please create a phased implementation plan with code for each phase.

10.3 Prompt Template for Debugging

I'm getting [describe error or unexpected behavior] in [file path].

EXPECTED: [what should happen]

ACTUAL: [what is happening]

CONSOLE ERRORS: [paste any console errors]

Please add strategic console.log statements to help trace the issue.

Focus on: [data flow / rendering / state / API response].

10.4 Commit Message Convention

Format: `type(scope): brief description`

Type	When to Use
feat	New feature or component
fix	Bug fix
style	CSS/styling changes only
refactor	Code restructuring (no behavior change)
chore	Config, deps, tooling
docs	Documentation only
test	Adding or fixing tests

Examples:

```
feat(home): add PopularTourSection with tour card grid
fix(auth): resolve infinite redirect on expired token
style(button): update hover transition to 300ms ease-in
refactor(redux): extract tour async thunks to tourSlice
chore(deps): add redux-persist for auth state persistence
```

11. TESTING & DEBUGGING PROCEDURES

11.1 Manual Testing Checklist (Per Component)

Before pushing any component, verify all of the following:

- [] All variant styles render correctly
- [] Hover states work (colors change on mouse over)
- [] Active/pressed states work (click and hold shows different color)

- [] Disabled state prevents click and visually dims the element
- [] Component resizes properly when browser window changes width
- [] No errors or warnings in browser console
- [] Text content, spacing, and sizing match Figma design
- [] Fonts render as Raleway (check in DevTools → Computed → font-family)
- [] Colors match design tokens (inspect element → check computed colors)

11.2 Debug Strategy (Step by Step)

When something doesn't work:

1. **Open browser DevTools Console** — read the error message carefully
2. **Check the component's props** — add `console.log('Props:', props)` at the top of the component
3. **Check Redux state** — open Redux DevTools extension → inspect the relevant slice
4. **Check Network tab** — verify API calls are being made, check response status and body
5. **Isolate the component** — temporarily render it alone in App.jsx to rule out parent issues
6. **Check import paths** — wrong paths are the #1 cause of "component not rendering"
7. **Check Tailwind class names** — Tailwind v4 syntax may differ from v3 tutorials online

11.3 Common Issues Reference

Symptom	Likely Cause	Fix
Component not rendering at all	Wrong import path or wrong export	Check file path casing, check <code>export default</code>
Styles not applying	CSS variable not in @theme block	Add the variable to App.css @theme
Hover effect not working	Missing <code>hover:</code> prefix in Tailwind class	Add <code>hover:bg-secondary-normal-hover</code>
Redux state is <code>undefined</code>	Slice not added to store combineReducers	Add reducer to redux/store.js
API call returns 401	Token not attached or expired	Check api.js interceptor, check cookie name
Infinite re-render loop	useEffect missing dependency array	Add <code>[]</code> for mount-only, or proper deps
<code>VITE_API_BASE_URL</code> is undefined	Missing <code>VITE_</code> prefix or .env not loaded	Restart dev server after .env changes
Page shows blank after route change	Missing <code><Outlet /></code> in layout	Ensure TouristLayout renders <code><Outlet /></code>

12. GIT WORKFLOW & VERSION CONTROL

12.1 Branch Strategy

```
main (protected – production-ready)
  ■■■ develop (integration branch)
    ■■■ feat/home-page
    ■■■ feat/tours-page
    ■■■ feat/blog-page
    ■■■ feat/contact-page
    ■■■ feat/auth-flow
    ■■■ feat/tour-partners
    ■■■ feat/gallery
    ■■■ feat/admin-dashboard
    ■■■ fix/auth-token-refresh
    ■■■ fix/pagination-off-by-one
```

12.2 Git Rules

1. **Never push directly to `main` or `develop`** — always create a feature branch
2. **Branch naming:** `feat/short-name` or `fix/short-name`
3. **Commit often, push often** — small logical units. This lets you easily revert.
4. **Before pushing:** always run `npm run lint` and `npm run build`
5. **Pull from `develop` frequently** — run `git pull origin develop` before starting new work
6. **Pull request required** — all code goes through PR review before merging to `develop`

12.3 How to Revert When Things Break

```
# See the last 10 commits
git log --oneline -10

# Option A: Undo the last commit but keep changes in working directory
git reset --soft HEAD~1

# Option B: Undo the last commit AND discard all changes
git reset --hard HEAD~1

# Option C: Revert a specific commit (creates a new "undo" commit – safest)
git revert <commit-hash>

# Option D: Go back to a known good state
git stash          # save current work
git checkout <commit-hash> -- src/components/ui/Button.jsx # restore one file
```

13. SECURITY & PERFORMANCE STANDARDS

13.1 Security Checklist

- **Never hardcode API keys, tokens, or secrets** in source code. Use `.env` with `VITE_` prefix.
- **Store auth tokens in `httpOnly` cookies** where possible (the project uses `universal-cookie`).
- **Never use `dangerouslySetInnerHTML`** with any user-generated content.
- **Validate all form inputs** client-side before submission using `utils/validators.js`.
- **Use HTTPS** for all API endpoints in production.

- **Remove all `console.log` debug statements** before pushing code.
- **Do not log sensitive data** (passwords, tokens, personal info) even during debugging.

13.2 Performance Checklist

- **Lazy-load pages** for code splitting:

```
const HomePage = React.lazy(() => import('../pages/tourist/HomePage'));  
// Wrap in <Suspense fallback={<Loader />}>
```
- **Optimize images:** Use WebP format, set explicit `width` and `height` attributes.
- **Memoize expensive renders:**
 - `React.memo()` for pure display components like cards
 - `useMemo()` for computed/filtered lists
 - `useCallback()` for handlers passed as props to memoized children
- **Debounce search/filter inputs** using the `useDebounce` custom hook.
- **Paginate all API calls** — never load all items at once.
- **Avoid inline object/array creation in JSX** — this creates new references every render:

```
// BAD – creates new object every render  
<Card style={{ padding: 10 }} />  
  
// GOOD – define outside or use useMemo  
const cardStyle = useMemo(() => ({ padding: 10 }), []);  
<Card style={cardStyle} />
```

13.3 Accessibility Standards

- Use semantic HTML elements: `<nav>`, `<main>`, `<section>`, `<article>`, `<button>`, `<form>`
- All `<img>` tags must have meaningful `alt` text
- All icon-only buttons must have `aria-label`
- Interactive elements must be keyboard-accessible (focusable, activatable with Enter/Space)
- Maintain WCAG AA color contrast (the defined palette already meets this)
- Form inputs must have associated `<label>` elements
- Use `role="navigation"` and `aria-label` on nav components (existing code already does this)

14. CODE REVIEW FRAMEWORK (FOR SENIOR DEVELOPER)

14.1 How to Conduct a Review

When a developer pushes code and marks their task as "In Review":

1. **Pull their branch locally:** `git checkout feat/their-branch && git pull`
2. **Run the project:** `npm run dev` — verify it builds without errors
3. **Run the linter:** `npm run lint` — should have zero errors
4. **Open the browser** and visually inspect the implemented feature
5. **Walk through the checklist below** marking each item PASS / FAIL / N/A

-
6. **If any FAIL items:** Create a review comment describing what needs to change
 7. **If all PASS:** Approve and merge to develop

14.2 Review Checklist

Use this for every pull request. Mark each item: **P** (Pass), **F** (Fail), **N/A**.

A. FILE STRUCTURE & NAMING

#	Check	P/F/NA
A1	File is in the correct directory per SOP Section 3.1	
A2	File name follows PascalCase (components) or camelCase (utils/services)	
A3	Import order follows convention: React → Redux → Services → Hooks → Components → Utils → Assets	
A4	No unused imports	
A5	Component name matches file name	

B. COMPONENT PATTERNS

#	Check	P/F/NA
B1	Uses <code>React.forwardRef</code> wrapper	
B2	Uses <code>classNames()</code> utility (no manual string concatenation for classes)	
B3	Style variants defined as plain objects (not inline conditionals)	
B4	<code>...props</code> spread on root element	
B5	<code>className</code> is the last argument in <code>classNames()</code> call	
B6	<code>displayName</code> set on forwardRef components	
B7	Props have sensible default values	
B8	Boolean props use <code>is/has/should</code> prefix	
B9	Event handlers use <code>handle</code> prefix internally, <code>on</code> prefix for props	

C. TAILWIND & DESIGN SYSTEM

#	Check	P/F/NA
C1	Uses design system color tokens (not hardcoded hex values like <code>#7b2cbf</code>)	
C2	Uses design system typography tokens (not hardcoded <code>text-[16px]</code> where a token exists)	
C3	Hover state implemented with <code>hover:</code> prefix	
C4	Active state implemented with <code>active:</code> prefix	
C5	Disabled state visually distinct (dimmed, cursor-not-allowed)	
C6	Transitions use <code>transition-all duration-300 ease-in</code>	
C7	No inline styles except for CSS variable references	
C8	Responsive classes present where relevant	

D. REDUX & STATE MANAGEMENT

#	Check	P/F/NA
D1	Slice follows the template pattern (initialState with items, isLoading, error)	
D2	Async thunks use <code>createAsyncThunk</code> with <code>rejectWithValue</code>	
D3	Custom hook wraps all slice state and dispatch calls	
D4	Components use custom hooks (not direct <code>useSelector/useDispatch</code>)	
D5	Loading state shows a loader/skeleton in the UI	
D6	Error state shows an error message in the UI	
D7	Empty state handled (when data array is empty)	

E. API INTEGRATION

#	Check	P/F/NA
E1	Service uses the centralized <code>api.js</code> Axios instance	

#	Check	P/F/NA
E2	No hardcoded API URLs (uses <code>import.meta.env.VITE_API_BASE_URL</code>)	
E3	Error handling present in thunks (try/catch with <code>rejectWithValue</code>)	
E4	No sensitive data logged to console	
E5	API parameters properly sanitized / validated	

F. SECURITY & PERFORMANCE

#	Check	P/F/NA
F1	No hardcoded secrets, API keys, or tokens in source code	
F2	No <code>dangerouslySetInnerHTML</code> with user content	
F3	No <code>console.log</code> debug statements left in code	
F4	No unnecessary re-renders (check <code>useEffect</code> dependency arrays)	
F5	Large lists paginated (not loading all at once)	
F6	Images have explicit width/height or aspect-ratio	
F7	Form inputs validated before API submission	

G. CODE QUALITY

#	Check	P/F/NA
G1	No duplicated code (reuses existing components/utilities)	
G2	Component is focused — does one thing well	
G3	Props interface is clean and well-named	
G4	No overly complex nested ternaries (extract to variables or functions)	
G5	Comments explain "why" not "what" (code should be self-documenting)	
G6	No TypeScript/ESLint warnings when running <code>npm run lint</code>	

14.3 Review Feedback Format

When leaving feedback on a failed item, use this format:

```
[FAIL - B2] File: src/components/cards/PopularTourCard.jsx, Line 15
ISSUE: Class names concatenated manually with template literals instead of classNames()
EXPECTED: classNames(baseStyles, variants[variant], className)
ACTUAL: `${baseStyles} ${variant === 'active' ? 'bg-purple' : ''}`
ACTION: Refactor to use classNames() utility per SOP Section 5.2
```

15. FIGMA DESIGN REFERENCE GUIDE

15.1 How to Access the Figma File

Figma File: Elysium Tours

URL: <https://www.figma.com/design/1McsqvfbvdyAS1EqMyeAdO>

Last Modified: February 10, 2026

15.1.1 Generating a Figma Personal Access Token

If you need to access the Figma API programmatically (for scripts, exports, or automation), you must generate a Personal Access Token. Follow these steps exactly:

1. **Open Figma** in your browser or the desktop app.
2. **Click the Figma icon** (top-left corner of the screen) to open the main dropdown menu.
3. In the dropdown, click **"Help and account"**.
4. Click **"Account settings"** — this opens your account settings page in the browser.
5. On the Account Settings page, click the **"Security"** tab in the left sidebar.
6. **Scroll down** to the **"Personal access tokens"** section at the bottom of the Security page.
7. Click **"Generate new token"**.
8. Give the token a descriptive name (e.g., `elysium-dev-token`).
9. **Set the required scopes.** For this project you need at minimum:
 - `file_content:read` — required to read file data, nodes, and export images via the API
 - Optionally `file_content:write` if you need to post comments
10. Click **"Generate token"**.
11. **Copy the token immediately** — Figma only shows it once. Store it securely (e.g., in your `.env` file as `VITE_FIGMA_TOKEN` or in a password manager).

Important: Without the `file_content:read` scope, API calls to `/v1/files/` and `/v1/images/` will return a 403 error with "Invalid scope(s)". If you get this error, regenerate the token with the correct scopes.

15.2 Page Structure in Figma

Figma Page	Purpose	Use For
Thumbnail	File cover image	Ignore
Design Components	Color palette, Icons, Grid, Typography, Color Tokens, Logo	Reference for design system tokens
Design Components - Advance	Navbar variants, Button variants, Image components	Reference for component states
Designs Inspo	Inspiration screenshots	Ignore (reference only)
Design-Iterations	Early drafts — Home, Tours, Cards, etc.	Ignore (superseded by Main Designs)
Main Designs	Final production designs	USE THIS PAGE FOR ALL IMPLEMENTATION
Tablet mode	Empty	Future work
Mobile Mode	Empty	Future work

15.3 Main Design Frames — Node ID Reference

Use these node IDs when referencing designs in Claude prompts or for Figma API access.

Full Pages

Page	Figma Node ID	Dimensions
Home	134:1481	1728 x 7760
Tours	186:1274	1728 x 6410
Blog	244:1018	1728 x 8565
Contact Us	370:1829	1728 x 5359
Not Found (404)	395:1828	1728 x 1169
Tour Partners	772:10493 / 969:6651	1728 x 10881
Single Blog Post	295:6204	1728 x 7596
Gallery - All	564:2376	1728 x 11656

Blog Sub-Pages

Page	Figma Node ID
Travel Guide Explore All	290:4375
Destination Highlight Explore All	295:4951
Behind the Scenes Explore All	295:5292 / 295:5603
Partner Spotlight	295:5904
Behind the Scenes with Guides	290:3106
Diaspora Travel Stories	290:3602 / 290:3856

Page	Figma Node ID
Festival Calendar	290:4115
Destination Highlights	290:2733
Travel Guides Tips	290:2366

Tour Partner Profile Sub-Pages

Page	Figma Node ID	Dimensions
Tour Sites & Events	946:6216	1728 x 3937
Accommodation	946:6956	1728 x 3937
Transportation	946:7301	1728 x 3937
Tour Guides	946:8003	1728 x 4335
Restaurants & Dining	946:8313	1728 x 3937
Photo & Videographers	946:8693	1728 x 3934
Insurance & Other Services	946:9047	1728 x 3937

Gallery Sub-Pages

Page	Figma Node ID
Gallery - Destination	616:4560 / 618:8550
Gallery - Activities	616:5229
Gallery - Nature	616:5572
Gallery - Culture	616:5914
Gallery - Video	616:6236 / 616:6874
Gallery - Partner	616:6555
Gallery - Travel Highlights	617:7189
Image Viewer - Default	627:10752
Image Viewer - Expanded	751:9961
Video Section	627:11277
Video Playing	627:12888
Upload Modal	690:4305
Share Photo Form	726:4397

UI Component Frames

Component	Figma Node ID
Breadcrumb	239:1591 / 239:1605
Search Icon	256:1811

Component	Figma Node ID
Filters	256:1844
Headline component	290:2001
Pagination	295:4844 / 295:4935
Toast Messages	419:2069 (section)
Toast - Default	415:1863
Toast - Error	415:2125
Toast - Warning	415:2128
Sort Dropdown	574:2782
Skeleton Loaders	1132:14569 / 1132:15340
Video Speed Control	772:10455
Maps Component	381:3011
File Upload - Drag	740:5346
File Upload - Browse	743:5548
File Upload - Failed	743:5764
Date Picker	1101:24539
Time Picker	1143:24558
Input Error States	735:4650 / 743:8544
Filter Panels	1009:7515 / 1076:12787 / 1108:27116
Location Filter	1080:14301 / 1101:13686
Sort Dropdowns	1175:17053 / 1215:19221 / 1229:23259
Menu/Navigation	883:5546
Tour Partners Dropdown	883:5698
Partner Highlights Cards	936:5795 / 946:5841 / 946:5903 / 946:6175

15.4 How to Use Figma Node IDs

To view a specific frame in Figma, construct the URL:

`https://www.figma.com/design/1McsqvfbvdyAS1EqMyeAd0?node-id=NODE_ID`

Replace `NODE_ID` with the value (e.g., `134:1481` becomes `134-1481` in the URL with a dash, or use `134%3A1481` URL-encoded).

To fetch design details programmatically (for advanced reference):

```
curl -H "X-Figma-Token: YOUR_TOKEN" \
  "https://api.figma.com/v1/files/1McsqvfbvdyAS1EqMyeAd0/nodes?ids=134:1481&depth=3"
```

16. APPENDICES

Appendix A: CSS Custom Property Quick Reference

All tokens available in `App.css @theme` block. Use in Tailwind as class names.

Colors: `bg-{primary|secondary|tertiary}-{light|normal|dark}-{default|hover|active}`
`text-{primary|secondary|tertiary}-{light|normal|dark}-{default|hover|active}`
`border-{primary|secondary|tertiary}-{light|normal|dark}-{default|hover|active}`

Typography: `font-raleway`
`text-[10px] font-[400-800] (sm)`
`text-[13px] font-[400-800] (med-small)`
`text-[16px] font-[400-800] (md)`
`text-[20px] font-[400-800] (semi-md)`
`text-[25px] font-[400-800] (High-md)`
`text-[31px] font-[400-800] (Display-small)`
`text-[39px] font-[400-800] (Display-md-small)`
`text-[49px] font-[400-800] (Display-md)`
`text-[56px] font-[400-800] (Display-xl)`
`text-[61px] font-[400-800] (Display-2xl)`
`text-[76px] font-[400-800] (Display-3xl)`

Radius: `rounded-[10px] (sm)`, `rounded-[40px] (md)`, `rounded-[100px] (lg)`
Shadow: `shadow`, `shadow-lg`

Appendix B: Useful VS Code Settings

```
// .vscode/settings.json
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "tailwindCSS.experimental.classRegex": ["classNames\\(([^)]*)\\)"],
  "emmet.includeLanguages": { "javascript": "javascriptreact" },
  "files.associations": { "*.jsx": "javascriptreact" }
}
```

Appendix C: Key Dependencies Cheat Sheet

Package	Import	Common Usage
<code>react-router-dom</code>	<code>import { useNavigate, useParams, useLocation, Link } from 'react-router-dom'</code>	Navigation and routing
<code>react-redux</code>	<code>import { useSelector, useDispatch, Provider } from 'react-redux'</code>	Redux access (use custom hooks instead)
<code>@reduxjs/toolkit</code>	<code>import { createSlice, createAsyncThunk, configureStore } from '@reduxjs/toolkit'</code>	Redux setup
<code>axios</code>	<code>import api from '../services/api'</code>	API calls (always use the configured instance)
<code>universal-cookie</code>	<code>import Cookies from 'universal-cookie'</code>	Cookie read/write

Package	Import	Common Usage
redux-persist	<code>import { persistStore, persistReducer } from 'redux-persist'</code>	State persistence

Appendix D: Glossary for Junior Developers

Term	What It Means
Component	A reusable piece of UI (like a Button or Card). In React, it's a function that returns JSX.
Props	Data passed into a component from its parent. Like function arguments.
State	Data that can change over time within a component or in Redux.
Redux Slice	A self-contained chunk of Redux state with its reducers and actions.
Thunk	An async function that can dispatch Redux actions (used for API calls).
Interceptor	Code that runs before every HTTP request or after every response (used for auth tokens).
Hook	A React function (starting with <code>use</code>) that lets you use state and lifecycle features.
forwardRef	A React wrapper that lets a parent component get a reference to a child's DOM element.
Tailwind utility class	A small CSS class that does one thing (e.g., <code>bg-red-500</code> sets background to red).
CSS custom property	A variable defined in CSS (e.g., <code>--color-primary: #fff</code>) that can be reused.
Variant	A visual variation of a component (e.g., a "primary" button vs an "outline" button).
Figma node ID	A unique identifier for any element in a Figma design file.
Layout	A wrapper component that provides shared UI (Navbar + Footer) around page content.
Outlet	A React Router placeholder where child route content gets rendered inside a layout.

END OF DOCUMENT

This SOP should be treated as a living document. Update it as patterns evolve, new components are added, or when team feedback suggests improvements.